

EJERCICIO PRÁCTICO

ARQUITECTO DE

SOLUCIONES

Autor: Paúl Jérez

Fecha de Elaboración: DMQ, 02 de Junio de 2026

Contenido

1.	Descripción del Ejercicio Práctico:	2
2.	Resumen Ejecutivo	4
3.	Requerimientos Funcionales y No Funcionales	5
4.	Arquitectura de alto nivel	6
4.1.	Principios de Diseño	6
5.	Modelo C4	6
3.1.	Modelo de Contexto (Nivel 1)	6
3.2.	Modelo de Contenedores (Nivel 2)	12
3.3.	Modelo de Componentes (Nivel 3)	16
6.	Arquitectura Frontend SPA y Móvil	21
7.	Arquitectura de Autenticación y Autorización	23
8.	Arquitectura de Integración	26
9.	Persistencia y Auditoria	27
10.	Notificaciones	29
11.	Infraestructura en la nube	30
12.	Seguridad y cumplimiento Normativos	31
13.	Gobernanza de APIs y Microservicios	32
14.	Patrones de integración y tecnologías	33
15.	Infraestructura en la nube (AWS)	36
16.	Plan de Migración Gradual	40
17.	Conclusiones	41
18.	Glosario de términos y siglas	43

1. Descripción del Ejercicio Práctico:

- Cree un documento PDF con la respuesta al ejercicio.
 - o El documento debe estar bien organizado y ser fácil de leer.
 - o Agregue imágenes para explicar los diagramas. Puedes usar herramientas como <http://draw.io/> para generar las imágenes.
 - o Cada uno de los diagramas de C4 es un entregable importante. Asegúrese de hacerlos correctos y detallados.
 - o Preste también atención a los otros diagramas requeridos.
 - o Asegúrese de añadir cualquier texto explicativo que considere necesario.

Suba el documento como respuesta a este ejercicio.
Asegúrese de subirlo dentro del tiempo de resolución.

Adicional, crear un repositorio público en Github y subir el PDF a ese repositorio.
Colocar la URL al repositorio en los comentarios de este ejercicio:

Recomendaciones MUY valoradas:

Cada decisión arquitectónica en el diseño debe tener una justificación teórica (Mínimo 2 por cada una). Ejemplo: ¿Por qué se decidió por determinados patrones, tecnologías, componentes, protocolos, etc.? ¿Qué opciones evaluó?

Se debe incluir los siguientes diagramas en el diseño de la solución:

Diagrama de Contexto: Para usuarios no técnicos. Muestra sistemas internos y externos, actores clave, breves descripciones y conexiones explicativas.

Diagrama de Contenedores : Para técnicos. Representa aplicaciones, servicios, bases de datos y mensajería, incluyendo componentes cloud sin mucho detalle. Conexiones con descripciones breves.

Descripción del ejercicio:

La entidad BP ha contratado a un arquitecto de soluciones para crear un sistema de banca por internet donde los usuarios puedan acceder al historial de movimientos, realizar transferencias y pagos entre cuentas propias e interbancarias.

La información del cliente provendrá de dos sistemas: una plataforma Core con datos básicos y otra independiente para información complementaria.

Se debe usar mínimo dos sistemas de notificaciones para avisar a los usuarios sobre movimientos realizados.

El sistema incluirá dos aplicaciones en el Front, una SPA y una aplicación móvil desarrollada en un Framework multiplataforma, solicitando mencionar dos opciones y justificar la elección.

Ambas aplicaciones autenticarán a los usuarios mediante el estándar OAuth2.0, y se pide recomendar el mejor flujo de autenticación según el estándar.

Finalmente, el sistema de Onboarding para nuevos clientes en la aplicación móvil debe considerar el reconocimiento facial como parte del flujo de autorización y autenticación, y se sugiere recomendar herramientas de la industria que realicen estas tareas y robustezcan la aplicación.

El sistema registra todas las acciones del cliente y utiliza un mecanismo de persistencia de información para clientes frecuentes. Se solicita proponer una alternativa basada en patrones de diseño que relacione los componentes necesarios para cumplir el objetivo.

El sistema obtiene los datos del cliente mediante una capa de integración compuesta por un API Gateway, que consume servicios según el tipo de transacción. Inicialmente, se mencionan tres servicios principales: consulta de datos básicos, consulta de movimientos y transferencias, con posibilidad de agregar más servicios para mejorar el rendimiento o la respuesta al cliente.

Consideraciones:

- Mencionar elementos normativos relevantes para entidades financieras (ley de datos personales, seguridad, etc.).
- Garantizar alta disponibilidad (HA), tolerancia a fallos, recuperación ante desastres (DR), seguridad y monitoreo, excelencia operativa y auto-healing.
- Posibilidad de incluir infraestructura en nube (Azure o AWS), garantizando baja latencia y presupuesto adecuado.
- Plantear una arquitectura desacoplada con elementos reusables y cohesionados.
- Desarrollar el modelo bajo modelo C4 (Modelo de Contexto, Modelo de aplicación o contenedor y componentes), describiendo hasta el modelo de componentes y modelando la infraestructura con la herramienta de preferencia.

Criterios de calificación

Se calificarán los siguientes elementos:

- Que la solución satisfaga los requerimientos y todo cuente con justificación
- Calidad y profundidad de los diagramas (Contexto, Contenedores y Componentes)
- Segmentación de Responsabilidades y Desacoplamiento
- Uso de patrones de arquitectura
- Integración con servicios externos
- Calidad de arquitectura de aplicación front-end y móvil
- Arquitectura de acceso a datos
- Conocimientos de Nube (AWS o Azure)
- Manejo de costos
- Arquitectura de Autenticación
- Arquitectura de Integración con Onboarding
- Diseño de Solución de Auditoría
- Conocimientos de regulaciones bancarias y estándares de seguridad
- Implementación de Alta Disponibilidad y Tolerancia a Fallos
- Implementación de Monitoreo.

2. Resumen Ejecutivo

La entidad financiera BP requiere el diseño de un sistema de banca por Internet que permita a sus clientes:

- Consultar historial de movimientos.
- Realizar transferencias y pagos entre cuentas propias e interbancarias.
- Recibir notificaciones de operaciones realizadas.
- Incorporar un proceso de Onboarding digital con reconocimiento facial.

El sistema propuesto se construirá bajo un modelo nube-nativa, desacoplado, resiliente, tolerante a fallos, alta disponibilidad (HA), recuperación ante desastres (DR) y estabilidad siguiendo el estándar Modelo C4 para esta documentación arquitectónica.

Esta solución va a contemplar:

Capa	Descripción
Frontend	Una SPA (Angular/React) y una aplicación móvil multiplataforma (Flutter o React Native).
Backend	Microservicios expuestos mediante un API Gateway , integrados con el Core bancario y sistemas complementarios.
Autenticación	Estándar OAuth2.0 , donde se usará el OpenID Connect flujo Authorization Code con PKCE para mayor seguridad en aplicaciones móviles.
Onboarding	Reconocimiento facial con herramientas líderes de la industria (ej. Amazon Rekognition , Azure Cognitive Services , FaceTec).
Notificaciones multicanal (email/SMS)	Alertas oportunas de movimientos sensibles. Cumple regulaciones de seguridad y notificación financiera.
Persistencia y Auditoría	Uso de patrones CQRS + Event Sourcing para trazabilidad y almacenamiento de clientes frecuentes.

Los beneficios clave que incluyen:

- ✓ Experiencia de usuario optimizada mediante aplicaciones SPA y móvil con notificaciones multicanal.
- ✓ Reducción de costos operativos gracias al uso de servicios cloud gestionados en Azure/AWS.
- ✓ Cumplimiento normativo con estándares internacionales (Ley de Protección de Datos, PCI DSS, ISO 27001).
- ✓ Excelencia operativa mediante monitoreo proactivo, recuperación ante desastres y auto-healing.

3. Requerimientos Funcionales y No Funcionales

La entidad BP cuenta actualmente con un Core Bancario Tradicional que gestiona la información básica de los clientes. Para complementar este sistema, se integra un sistema independiente de información adicional, lo que plantea la necesidad de una **arquitectura híbrida** que permita la coexistencia y orquestación de ambos entornos.

Está basada en microservicios, escalabilidad, resiliencia, tolerancia a fallo, HA, DRP, seguridad, APIs y eventos, lo hemos dividido en requerimientos Funcionales y No Funcionales, el cual detallamos a continuación:

Requisito Funcional	Justificación
Integración con Core Bancario Tradicional	Permite acceder a datos básicos del cliente (cuentas, saldos, información principal). Se mantiene la continuidad operativa con sistemas legacy.
Consulta de historial de movimientos	Acceso completo y actualizado a operaciones financieras, cumpliendo con ISO 20022 para estandarización de mensajes y trazabilidad.
Transferencias internas y externas.	Ejecución de pagos nacionales e interbancarios, garantizando cumplimiento de KYC/AML y regulaciones locales de la Superintendencia de Bancos.
Pagos electrónicos interbancarios	Uso de protocolos seguros (TLS 1.3, cifrado AES-256) y alineación con PCI DSS para protección de datos financieros.
Notificaciones en tiempo real	Integración con al menos dos sistemas (ej. Twilio SMS, Firebase Push) para redundancia y entrega garantizada.
Onboarding digital con biometría facial	Autenticación robusta mediante reconocimiento facial con herramientas líderes (ej. Azure Cognitive Services, Amazon Rekognition), cumpliendo con GDPR/Habeas Data .
Registro y auditoría de acciones	Implementación de patrones CQRS + Event Sourcing para trazabilidad, auditoría y cumplimiento normativo.

Requisito No Funcional	Justificación
Alta disponibilidad (HA)	SLA $\geq 99.9\%$, con despliegue en múltiples zonas de disponibilidad en AWS/Azure .
Tolerancia a fallos y DRP	Replicación activa-activa y planes de recuperación ante desastres, siguiendo ISO 22301 .
Seguridad	Autenticación OAuth2.0 con flujo Authorization Code + PKCE , cifrado en tránsito/reposo AES-256 , TLS 1.3 , cumplimiento de ISO 27001 y PCI DSS .
Escalabilidad, bajo acoplamiento, alta cohesión	Arquitectura cloud-native con autoescalado horizontal, desacoplamiento mediante microservicios y API Gateway.
Observabilidad, trazabilidad y auditoría	Uso de Prometheus, Grafana, ELK Stack, OpenTelemetry para métricas, logs y trazas distribuidas.

4. Arquitectura de alto nivel

4.1.Principios de Diseño

La solución propuesta se fundamenta en los siguientes principios arquitectónicos:

Desacoplamiento

- Separación clara entre frontend, backend y capa de integración.
- Uso de un **API Gateway** para aislar servicios y facilitar la evolución independiente.

Escalabilidad

- Arquitectura **cloud-native** con microservicios desplegados en contenedores (Kubernetes/EKS/AKS).
- Autoescalado horizontal para soportar picos de demanda en transferencias y consultas.

Alta Disponibilidad (HA)

- Despliegue en múltiples zonas de disponibilidad en la nube (AWS/Azure).
- Balanceadores de carga y replicación activa-activa para garantizar continuidad.

Recuperación ante Desastres (DR)

- Estrategia de replicación geográfica y backups automatizados.
- Plan de recuperación siguiendo estándares **ISO 22301**.

Seguridad

- Autenticación y autorización mediante **OAuth2.0** con flujo **Authorization Code + PKCE**.
- Cifrado de datos en tránsito (TLS 1.3) y en reposo (AES-256).
- Cumplimiento de PCI DSS, ISO 27001 y normativa local de protección de datos.

5. Modelo C4

3.1.Modelo de Contexto (Nivel 1)

El Diagrama de Contexto representa la visión más macro del sistema de banca por internet de BP. Está orientado a usuarios no técnicos, mostrando los actores principales, sistemas internos y externos, y las interacciones clave.

En este nivel se busca responder:

- ¿Quiénes son los actores que interactúan con el sistema?
- ¿Qué sistemas externos se integran?
- ¿Cuáles son las conexiones críticas y su propósito?

El sistema se concibe como un ecosistema híbrido, donde el Core Bancario Tradicional provee datos básicos de clientes, mientras que un Sistema Complementario aporta

información adicional. Ambos se integran mediante un API Gateway, que expone servicios de consulta, transferencias y pagos.

Los usuarios acceden al sistema a través de dos aplicaciones:

- Una **SPA (Single Page Application)** para web.
- Una **Aplicación Móvil multiplataforma** (Flutter o React Native).

La autenticación se realiza mediante **OAuth2.0 (Authorization Code + PKCE)**, garantizando seguridad en aplicaciones móviles y web.

Los eventos críticos (transferencias, pagos, movimientos) generan **notificaciones redundantes** vía SMS y Push, asegurando la entrega. Además, el sistema se integra con proveedores de **biometría facial** para el proceso de Onboarding digital

Primeramente, vamos a detallar una matriz del diagrama de Contexto mediante las siguientes columnas con los nombres: Actor/Sistema, Rol Funcional , Tipo de interacción y las notas estratégicas.

Actor / Sistema	Rol Funcional	Tipo de Interacción	Notas Estratégicas
Cliente (usuario final)	Consumidor de servicios bancarios	Interacción vía SPA y Mobile App	Experiencia fluida y segura; clave para fidelización.
Core Bancario Tradicional	Fuente de datos básicos	Exposición de cuentas, saldos, datos principales	Continuidad operativa con sistemas legacy.
Sistema Complementario	Fuente de datos adicionales	Perfil extendido, preferencias, información complementaria	Mejora la personalización y segmentación de servicios.
API Gateway	Orquestador de servicios	Enrutamiento y consumo de APIs	Centraliza consumo; facilita monitoreo y seguridad.
SPA (Web App)	Frontend web	Acceso a consultas y transferencias	Experiencia moderna, responsive y segura.
Mobile App (Flutter/React Native)	Frontend móvil	Acceso a consultas, transferencias y Onboarding	Multiplataforma; integración con biometría facial.
Servicio de Autenticación (OAuth2.0)	Autenticación y autorización	Validación de credenciales y tokens	Flujo Authorization Code + PKCE; seguridad reforzada.
Sistema de Notificaciones (SMS/Push)	Comunicación con clientes	Envío de alertas y confirmaciones	Redundancia para garantizar entrega.
Proveedor Biométrico	Validación de identidad en onboarding	Autenticación y autorización facial	Cumple estándares de seguridad y privacidad (GDPR, Habeas Data).

(Reconocimiento Facial)			
Reguladores / Entidades Financieras	Supervisión y cumplimiento	Auditoría y reportes	Cumplimiento de normativas locales e internacionales.
Fintechs / Terceros	Consumidores de APIs abiertas	Interacción vía Open Banking	Expande el ecosistema financiero bajo estándares BIAN.
Sistema de Monitoreo	Observabilidad	Métricas, logs, alertas	Garantiza SLA y resiliencia.
Sistema de Auditoría	Registro y trazabilidad de eventos	Integración con microservicios internos	Cumple con normativas financieras y de seguridad.

Ilustración 1: Matriz del Diagrama de Contexto.

Beneficios para la Entidad Bancaria

- **Seguridad:** La autenticación vía OAuth2.0 es clave para proteger los datos del cliente y cumplir con regulaciones como GDPR o PCI-DSS.
- **Escalabilidad:** El uso de API Gateway permite agregar nuevos servicios sin afectar la arquitectura principal.
- **Experiencia del Usuario:** Las notificaciones (SMS y Push) deben ser oportunas y relevantes para mantener al cliente informado y comprometido.
- **Modularidad:** Separar el Core Bancario del Sistema Complementario permite una evolución independiente de cada componente.

Diagrama de Contexto del Sistema de Banca por Internet

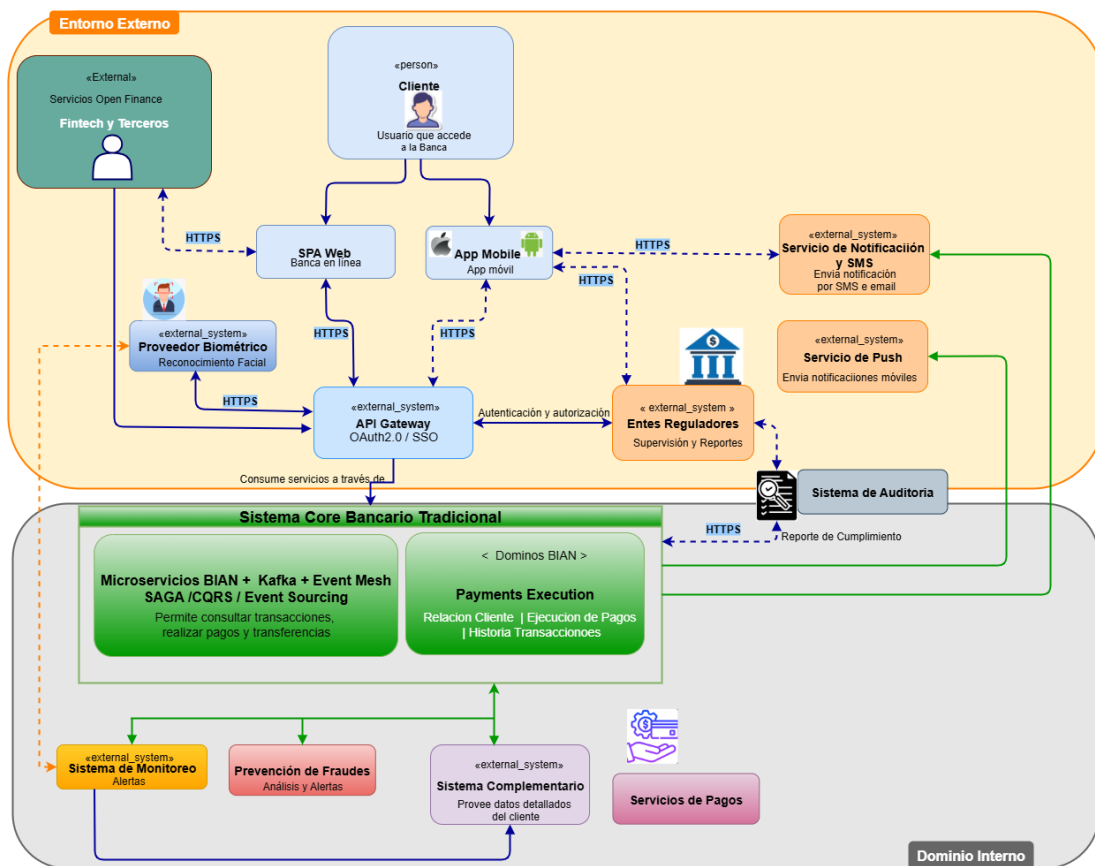


Ilustración 2.:Diagrama de Contexto entidad Bancaria

El Diagrama de Contexto del Sistema de Banca por Internet muestra la visión macro del ecosistema, diferenciando claramente el Entorno Externo y el Dominio Interno.

Una vez autenticado ,el cliente , el sistema se conecta con varios servicios, que cumplen las siguientes funciones:

Entorno Externo

- ✓ **Cliente (Usuario Final):** accede al sistema mediante la **SPA Web** y la **App Móvil**, interactuando con el API Gateway.
- ✓ **Fintech y Terceros (Open Finance):** consumen APIs abiertas, ampliando el ecosistema financiero.
- ✓ **Proveedor Biométrico:** integra reconocimiento facial para el proceso de Onboarding digital, reforzando seguridad y cumplimiento normativo.
- ✓ **API Gateway (OAuth2.0 / SSO):** gestiona autenticación, autorización y enrutamiento de servicios, asegurando control centralizado.
- ✓ **Entes Reguladores:** supervisan operaciones y reciben reportes de cumplimiento.
- ✓ **Servicios de Notificación (SMS y Push):** garantizan comunicación redundante y en tiempo real con el cliente.

Dominio Interno

- ✓ **Core Bancario Tradicional:** mantiene datos básicos de clientes y operaciones.
- ✓ **Microservicios BIAN + Kafka + Event Mesh (SAGA / CQRS / Event Sourcing):** permiten consultas de movimientos, ejecución de pagos y transferencias con resiliencia y trazabilidad.
- ✓ **Sistema Complementario:** aporta información adicional del cliente para enriquecer la experiencia y personalización.
- ✓ **Prevención de Fraudes:** analiza transacciones y genera alertas para mitigar riesgos.
- ✓ **Sistema de Monitoreo:** capturas métricas, logs y alertas para garantizar SLA y resiliencia.
- ✓ **Sistema de Auditoría:** registra todas las acciones y genera reportes de cumplimiento normativo.
- ✓ **Servicios de Pagos:** gestionan la ejecución de transferencias y pagos interbancarios.

Justificación Teórica :

1. Uso de OAuth2.0 para Autenticación, se eligió por tres puntos:

- ✓ **Estándar internacional y robusto:** OAuth2.0 es ampliamente adoptado en sistemas bancarios, fintechs y plataformas digitales, garantizando interoperabilidad y compatibilidad con proveedores externos (Google, Apple, Microsoft).
- ✓ **Separación de responsabilidades:** Delegar la autenticación a un servicio especializado reduce riesgos en el sistema principal y facilita la integración con Identity Providers (IdP).
- ✓ **Escalabilidad y seguridad avanzada:** Permite implementar flujos seguros como Authorization Code + PKCE, ideales para aplicaciones móviles y SPA, evitando ataques de interceptación de tokens.

Opciones evaluadas:

- ✓ **Autenticación básica (usuario/contraseña):** descartada por ser menos segura, no escalable y vulnerable a ataques de fuerza bruta.
- ✓ **JWT sin OAuth:** viable, pero menos flexible para delegar autenticación a terceros y sin soporte nativo para flujos de autorización complejos.

2. Uso de API Gateway como punto de entrada cuyo objetivo son:

- ✓ **Centralización del acceso:** El API Gateway controla, monitorea y asegura todas las llamadas a servicios internos desde un único punto, aplicando políticas de seguridad (rate limiting, WAF, validación de tokens).
- ✓ **Escalabilidad y mantenimiento:** Facilita la gestión de múltiples microservicios internos sin exponer directamente sus endpoints, soportando versionamiento y gobernanza de APIs.

- ✓ **Observabilidad y resiliencia:** Permite aplicar patrones como **Circuit Breaker**, **Retry** y **Fallback**, además de integrar métricas y trazas distribuidas para monitoreo.

Las opciones que se evaluaron son:

- ✓ **Acceso directo a microservicios:** descartado por falta de control, seguridad y dificultad de mantenimiento.
- ✓ **Bus de servicios (ESB):** considerado, pero menos ágil y flexible para arquitecturas modernas basadas en microservicios y eventos.

3. Uso de CQRS (Command Query Responsibility Segregation) + Event Sourcing

- ✓ **Separación de cargas de trabajo:** CQRS permite diferenciar entre operaciones de lectura (consultas de movimientos, saldos) y operaciones de escritura (transferencias, pagos). Esto optimiza el rendimiento y evita que procesos intensivos afecten la experiencia del cliente.
- ✓ **Trazabilidad y auditoría:** combinado con Event Sourcing, cada acción del cliente se registra como un evento inmutable. Esto garantiza transparencia, facilita auditorías y permite reconstruir el estado del sistema en cualquier momento.
- ✓ **Escalabilidad independiente:** las consultas pueden escalarse de forma distinta a las escrituras, lo que mejora la eficiencia en escenarios de alta demanda.

Opciones evaluadas:

- ✓ **Modelo CRUD tradicional:** descartado por no ofrecer trazabilidad completa ni escalabilidad diferenciada.
- ✓ **Bases de datos relacionales sin eventos:** viables, pero limitadas en auditoría y recuperación ante fallos.

4. Uso de Saga Pattern (Orquestación de transacciones distribuidas)

- ✓ **Consistencia eventual en operaciones críticas:** el patrón Saga permite manejar transacciones distribuidas (ej. transferencias interbancarias) mediante pasos coordinados y compensaciones en caso de error.
- ✓ **Resiliencia ante fallos:** evita bloqueos prolongados de recursos y reduce el riesgo de inconsistencias en operaciones multi-servicio.
- ✓ **Flexibilidad en integración:** se adapta a arquitecturas basadas en microservicios y mensajería (Kafka, Event Mesh), garantizando que cada servicio pueda ejecutar su parte de la transacción de forma autónoma.

Opciones evaluadas:

- ✓ **Transacciones distribuidas con 2PC (Two-Phase Commit):** descartadas por su complejidad y baja tolerancia a fallos en entornos distribuidos.

- ✓ **Orquestación manual de procesos:** considerada, pero poco escalable y difícil de mantener en arquitecturas modernas.

3.2. Modelo de Contenedores (Nivel 2)

En este modelo de Contenedores permite visualizar la arquitectura del sistema como un conjunto de bloques funcionales independientes pero interconectados. Esta estructura facilita la escalabilidad, la seguridad, el mantenimiento y la evolución tecnológica del sistema. A continuación, se presenta la justificación por Contenedor, junto con las herramientas típicas que se emplean en cada una.

Contenedor	Rol Funcional	Herramientas	Justificación Estratégica
SPA Web	Frontend para banca en línea	Angular / React	Experiencia moderna, responsive y segura.
App Móvil	Frontend móvil multiplataforma	Flutter / React Native	Onboarding digital con biometría facial; soporte iOS/Android.
API Gateway	Punto de entrada y seguridad	Kong / Apigee / AWS API Gateway	Centraliza acceso, aplica OAuth2.0, rate limiting y monitoreo.
Servicio de Autenticación	Validación de credenciales y tokens	OAuth2.0 + PKCE	Flujo seguro para SPA y Mobile App.
Microservicio de Consulta de Datos	Acceso a datos básicos del cliente	Java / Spring Boot + CQRS	Escalable y desacoplado; integra Core Tradicional.
Microservicio de Movimientos	Consulta de historial de transacciones	Node.js / Quarkus + Event Sourcing	Garantiza trazabilidad y auditoría.
Microservicio de Transferencias/Pagos	Ejecución de transferencias internas e interbancarias	Java / Saga Pattern + Kafka	Manejo de transacciones distribuidas con compensaciones.
Sistema Complementario	Información adicional del cliente	APIs REST / GraphQL	Mejora personalización y segmentación.
Base de Datos Transaccional	Persistencia de operaciones	Aurora PostgreSQL / DynamoDB	Alta disponibilidad y consistencia.
Sistema de Notificaciones	Comunicación redundante	Twilio SMS / Firebase Push	Entrega garantizada y baja latencia.
Sistema de Auditoría	Registro de acciones	Event Store + ELK Stack	Cumplimiento normativo y trazabilidad.
Sistema de Monitoreo	Observabilidad	Prometheus / Grafana / OpenTelemetry	Métricas, logs y alertas distribuidas.
Prevención de Fraudes	Análisis de transacciones	ML Models + Kafka Streams	Mitigación de riesgos en tiempo real.

Ilustración 3: Matriz del Diagrama de Contenedores.

Valor Estratégico para la Entidad Bancaria

- **Alta disponibilidad y escalabilidad automática** sin necesidad de gestionar servidores.
- **Costos optimizados** por uso real (pay-per-use), ideal para cargas variables.
- **Seguridad multicapa** con control de acceso, cifrado y monitoreo continuo.
- **Interoperabilidad** con sistemas legacy y proveedores externos.
- **Cumplimiento normativo** y trazabilidad para auditorías y reguladores.

Beneficios Estratégicos del Enfoque por Contenedores:

- **Modularidad:** Cada contenedor puede evolucionar de forma independiente, facilitando actualizaciones sin afectar el sistema completo.
- **Escalabilidad Horizontal:** Permite distribuir la carga entre múltiples instancias de cada servicio.
- **Seguridad Granular:** Se pueden aplicar políticas específicas por contenedor (por ejemplo, autenticación reforzada en el backend).
- **Observabilidad:** Herramientas como Cloudwatch, Grafana permiten monitorear el sistema en tiempo real.
- **Automatización y DevOps:** Facilita la integración con pipelines CI/CD y despliegues en contenedores mediante ECS Fargate.

Diagrama de Contenedores del Sistema de Banca por Internet

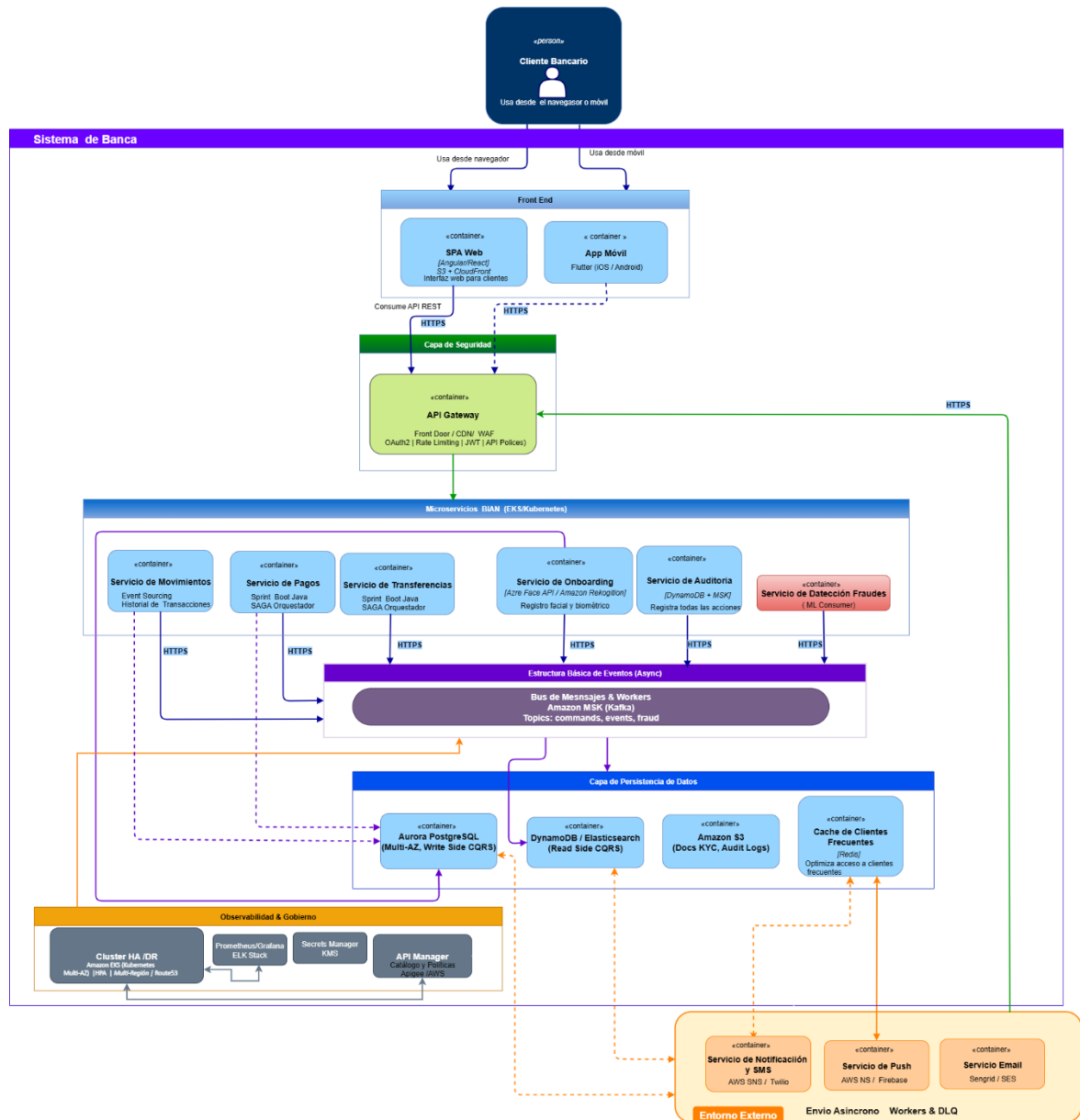


Ilustración 4: Diagrama de Contenedores

El Diagrama de Contenedores representa la arquitectura técnica del sistema de banca por internet, mostrando cómo los distintos componentes interactúan entre sí y con servicios externos. Este nivel detalla las aplicaciones, microservicios, bases de datos y mecanismos de seguridad y observabilidad que conforman la solución.

Cliente Bancario y Frontend:

El **Cliente Bancario** accede al sistema mediante dos canales digitales:

- ✓ **SPA Web (React/Angular):** interfaz web para consultas, transferencias y pagos.

- ✓ **App Móvil (Flutter):** aplicación multiplataforma para iOS y Android, que integra el proceso de Onboarding digital con reconocimiento facial.

Ambos frontends consumen servicios REST sobre **HTTPS**, garantizando comunicación segura y cifrada.

- ✓ Su interacción se realiza de forma segura mediante el protocolo **HTTPS** y autenticación con **OAuth2**.

Capa de Seguridad

El **API Gateway** (Kong, Apigee o WAF) actúa como punto de entrada único:

- Gestiona autenticación y autorización mediante **OAuth2.0 + PKCE**.
- Aplica políticas de seguridad, control de tráfico y versionamiento de APIs.
- Centraliza el monitoreo y auditoría de las llamadas a microservicios.

Este componente asegura el **desacoplamiento** entre los canales de acceso y los servicios internos.

Microservicios BIAN

La capa de negocio está compuesta por microservicios independientes, alineados con los dominios **BIAN**:

- **Servicio de Movimientos:** gestiona el historial de transacciones mediante **Event Sourcing**.
- **Servicio de Pagos y Transferencias:** ejecuta operaciones críticas con el patrón **Saga**, garantizando consistencia eventual y resiliencia.
- **Servicio de Onboarding:** maneja el registro de nuevos clientes y validación biométrica.
- **Servicio de Auditoría:** registra eventos y genera reportes de cumplimiento.
- **Servicio de Detección de Fraudes:** analiza transacciones en tiempo real usando **Kafka Streams** y modelos de machine learning.

Los microservicios se comunican de forma **asíncrona** a través del **Bus de Mensajes y Workers**, que gestionan comandos y eventos distribuidos.

Capa de Persistencia de Datos

La arquitectura de datos aplica el patrón **CQRS**:

- **Aurora PostgreSQL (Write Side):** maneja las operaciones transaccionales.
- **DynamoDB / Elasticsearch (Read Side):** optimiza las consultas y reportes.
- **Amazon S3:** almacena documentos KYC y archivos del cliente.
- **Cache de Clientes:** mejora la latencia y la respuesta del sistema.

Esta separación permite escalar lecturas y escrituras de manera independiente, garantizando rendimiento y trazabilidad.

Observabilidad y Gobierno

La capa de observabilidad asegura la estabilidad y cumplimiento normativo:

- **Prometheus / Grafana / ELK Stack:** monitoreo, métricas y trazas distribuidas.
- **Secret Manager / KMS:** gestión segura de claves y secretos.
- **API Manager:** control de versiones y políticas de ciclo de vida de APIs.
- **Cluster HA / DR:** garantiza alta disponibilidad y recuperación ante desastres.

Contenedor de lógica de negocio:

Este contenedor gestiona las reglas del negocio y coordina las operaciones internas.}, el cual es orquestado por Kubernetes. Está desarrollado con tecnologías como Node.js/Express o Spring Boot, y sigue el patrón de microservicios desacoplados llamado “Saga Pattern” y para la comunicación entre microservicio se usará Service Mesh (Istio).

API Backend: Expone la lógica del negocio y orquesta las operaciones, que expone endpoints RESTful. Se comunica con:

- ✓ **Servicio de Onboarding:** Registro de nuevos clientes y captura biométrica.
- ✓ **Servicio de Autenticación:** Maneja login, tokens y permisos.
- ✓ **Servicio de Auditoría:** Registra todas las acciones para trazabilidad.
- ✓ **Cache de Clientes Frecuentes:** Mejora el rendimiento en consultas repetidas.
- ✓ **Base de Datos de Transacciones:** Persistencia de operaciones bancarias.
- ✓ **Tecnología sugerida:** Node.js / Express/Spring Boot .
- ✓ **Patrón:** Microservicios desacoplados, con control de acceso vía API Gateway

API Gateway: Punto de entrada para servicios internos, controla acceso y seguridad.

- ✓ Controla el acceso a los microservicios, aplica políticas de seguridad y facilita la escalabilidad.
- ✓ Utiliza servicios como AWS API Gateway, Lambda y CloudFront para garantizar alta disponibilidad.

Infraestructura en la Nube: AWS como proveedor principal, con servicios gestionados para escalabilidad, alta disponibilidad y cumplimiento normativo.

3.3.Modelo de Componentes (Nivel 3)

En este Diagrama de Componentes describe la arquitectura interna del sistema de banca por Internet, desglosando los elementos que conforman cada microservicio y sus interacciones. Este nivel permite comprender cómo los módulos —controladores, servicios, repositorios y adaptadores— colaboran para garantizar seguridad, trazabilidad, resiliencia y cumplimiento normativo.

A diferencia de los niveles anteriores del modelo C4, este diagrama se centra en la **implementación lógica** y la **responsabilidad funcional** de cada componente dentro de la arquitectura de microservicios.

Su propósito es ofrecer una visión clara para los equipos de desarrollo, arquitectura y auditoría tecnológica, respondiendo preguntas clave como:

- ¿Cómo se estructuran internamente los microservicios y qué responsabilidades asume cada uno?
- ¿Qué patrones arquitectónicos (CQRS, Saga, Event Sourcing) se aplican para garantizar consistencia y disponibilidad?
- ¿Cómo se integran los mecanismos de auditoría, seguridad y monitoreo para cumplir con estándares financieros y regulatorios?

Componente	Rol Funcional	Herramientas / Tecnologías	Justificación Estratégica
Gateway / API Backend	-Punto de entrada y control de acceso	Kong / Apigee / OAuth2.0 + PKCE	Centraliza seguridad y autenticación; permite gobernanza y control de tráfico.
Controlador de Transacciones	Orquestador principal de pagos y transferencias	Spring Boot / REST / OpenAPI	Facilita la comunicación entre servicios; desacopla lógica de negocio del acceso a datos.
Servicio de Movimientos	Consulta historial de transacciones	Java / CQRS / PostgreSQL	Optimiza rendimiento separando lectura y escritura; garantiza trazabilidad.
Servicio de Transferencias	Ejecución de transferencias internas y externas	Node.js / CQRS / Kafka	Escalable y resiliente; soporta alta concurrencia y consistencia eventual.
Servicio de Pagos	Procesamiento de pagos interbancarios	Java / Saga Pattern / Event Mesh	Maneja transacciones distribuidas con compensaciones; evita bloqueos.
Servicio de Fraudes	Detección de anomalías y alertas	Python / ML Models / Kafka Streams	Analiza patrones en tiempo real; reduce riesgos operativos.
Servicio de Onboarding	Registro de nuevos clientes con biometría facial	React Native / Azure Face API / AWS Rekognition	Cumple normativas KYC y Habeas Data; mejora experiencia del usuario.
Repositorio de Clientes	Acceso a datos del cliente	Aurora PostgreSQL / DynamoDB	Alta disponibilidad y consistencia; integra datos del Core Bancario.
Repositorio de Auditoría	Registro de eventos y acciones	Event Store / ELK Stack	Cumple con PCI DSS e ISO 27001; permite auditorías completas.
Repositorio de Transacciones	Persistencia de operaciones financieras	PostgreSQL / CQRS	Garantiza integridad y recuperación ante fallos.
Servicio de Notificaciones	Envío de alertas SMS, Push y Email	Twilio / Firebase / AWS SNS / SES	Comunicación redundante y multicanal; mejora la experiencia del cliente.

API Manager Adapter	Gestión de APIs internas y externas	Apigee / AWS API Gateway	Facilita versionamiento y monitoreo de APIs; soporte para Open Banking.
Bus de Mensajes & Workers	Integración asincrónica entre servicios	Kafka / RabbitMQ / AWS SQS	Asegura resiliencia y desacoplamiento; soporta eventos distribuidos.
Auditoría & Logging Service	Registro centralizado de logs y métricas	Prometheus / Grafana / OpenTelemetry	Mejora observabilidad y cumplimiento normativo.
Secret Manager	Gestión segura de credenciales y llaves	AWS Secrets Manager / Azure Key Vault	Refuerza seguridad y evita exposición de datos sensibles.

Ilustración 5: Capas de los Componentes del Sistema Bancario.

Diagrama de Componentes del Sistema de Banca por Internet

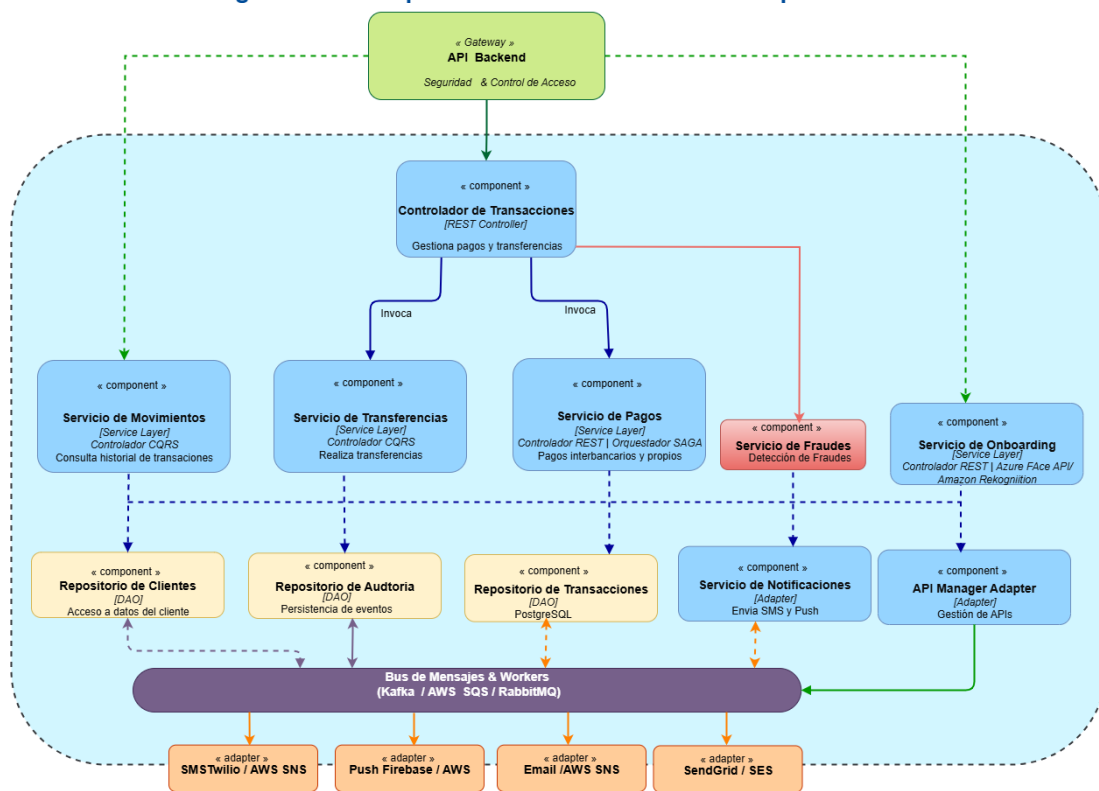


Ilustración 6: Diagrama de Componentes.

Contenedor Principal: API Backend

- **Responsabilidad:** Exponer endpoints RESTful para operaciones bancarias (transferencias, pagos, movimientos).
- **Tecnología sugerida:** Spring Boot / Node.js / .NET Core
- **Protocolo:** HTTPS + OAuth2 (seguridad en capa de transporte + autenticación federada)

- **Patrón arquitectónico:** Microservicio independiente, desacoplado del front-end y otros servicios

Capa de Seguridad y Acceso

- **Gateway / API Backend:** actúa como punto de entrada único al sistema.
 - Gestiona la **seguridad**, el **control de acceso** y la **autenticación** mediante OAuth2.0 o SSO.
 - Filtra y enruta las solicitudes hacia los controladores internos, garantizando trazabilidad y protección de datos.

Controlador de Transacciones [REST Controller]

- Implementa un **REST Controller** que gestiona pagos y transferencias.
- **Función:** Orquesta las operaciones de pagos y transferencias.
- **Invoca:** Servicios de dominio según el tipo de operación.
- **Seguridad:** Validación de token OAuth2, control de acceso por roles.

Capa de Persistencia

- ✓ **Repositorios DAO:** gestionan el acceso a datos del cliente, auditoría y transacciones.
- ✓ **PostgreSQL:** base de datos principal para operaciones transaccionales.
- ✓ **Event Store:** almacena eventos para trazabilidad y auditoría.

Esta capa implementa el patrón **CQRS**, separando lectura y escritura para optimizar rendimiento y consistencia.

Adaptadores Externos

- ✓ **Servicio de Notificaciones:** envía SMS y Push mediante **Twilio, Firebase o AWS SNS**.
- ✓ **API Manager Adapter:** gestiona políticas, versionamiento y monitoreo de APIs.
- ✓ Desacoplamiento de lógica de negocio y flexibilidad para cambiar proveedores.

Estos adaptadores permiten la integración con servicios externos y garantizan comunicación redundante con el cliente.

Bus de Mensajes y Workers

- ✓ Implementado con **Kafka / AWS SQS / RabbitMQ**.
- ✓ Facilita la comunicación asíncrona entre microservicios y la ejecución de tareas en segundo plano.

- ✓ Asegura consistencia eventual y tolerancia a fallos.

Entorno Externo

- ✓ **Twilio / SNS / Firebase / SES:** canales externos para notificaciones, correos y alertas.
- ✓ Permiten comunicación segura y confiable con los clientes y sistemas externos.

Servicios de Negocio (Service Layer)

Servicio	Función técnica	Interacciones clave
Movimientos	Consulta de historial de transacciones	Repositorio de Clientes
Transferencias	Validación de cliente y ejecución de transferencias	Repositorio de Clientes + Auditoría
Pagos	Pagos interbancarios + notificaciones	Servicio de Notificaciones
Riesgos	Evaluación del crediticio del cliente con las normativas.	API Manager + Notificaciones
Fraude	Anomalías de registro de transacciones por que pueden fraudes para para la entidad bancaria y el cliente.	API Manager + Notificaciones
Onboarding	Registro de nuevos clientes con biometría facial	Adapter Pattern + Azure Face API / Rekognition

- ✓ Estos servicios están desacoplados y se comunican mediante **mensajería asíncrona (Kafka, SQS, RabbitMQ)**, garantizando resiliencia y escalabilidad.
- ✓ **Patrón:** DDD (Domain-Driven Design), con separación clara entre lógica de negocio y persistencia
- ✓ **Protocolo interno:** Llamadas síncronas vía HTTP o asincronía con colas.

Adaptadores y Repositorios (DAO / Adapter Layer)

Componente	Rol técnico	Tecnología sugerida
Repositorio de Clientes	Acceso a datos de clientes	RDS PostgreSQL / DynamoDB
Repositorio de Auditoría	Registro de eventos críticos	CloudWatch Logs
Repositorio de Transacciones	Registro de movimientos	PostgreSQL
Servicio de Notificaciones	Envío de SMS y Push	AWS SNS / Pinpoint

- ✓ **Seguridad:** Logs cifrados, acceso controlado por IAM roles.
- ✓ **Cloud:** Uso de servicios gestionados en AWS para persistencia, mensajería y monitoreo.

Componentes Cloud Destacados

Componente	Servicio Cloud	Justificación
Persistencia	Amazon RDS / DynamoDB	Alta disponibilidad, backups automáticos

Auditoría	CloudWatch	Observabilidad, trazabilidad normativa
Notificaciones	SNS / Pinpoint	Escalabilidad en mensajería multicanal
Seguridad	AWS Cognito / IAM / Security Hub/ GuardDuty	Autenticación federada, gestión de claves

Justificación 1: Separación por capas + microservicios

- **Motivo:** Facilita mantenimiento, pruebas unitarias, escalabilidad por dominio
- **Evaluación:** Se comparó con monolitos y arquitecturas hexagonales. Se eligió microservicios con DDD para permitir evolución independiente de cada servicio (ej. pagos vs transferencias).
- **Beneficio:** Permite escalar servicios críticos sin afectar el resto del sistema y mejora la trazabilidad y el control normativo.

Justificación 2: Uso de adaptadores para notificaciones y auditoría

- **Motivo:** Desacoplar lógica de negocio de servicios externos (SMS, logs, push)
- **Evaluación:** Se consideró integración directa, pero se optó por adaptadores para facilitar pruebas, mocking y cambios de proveedor.
- **Beneficio:** Flexibilidad para cambiar de Pinpoint, o de CloudWatch sin afectar el core del sistema.

6. Arquitectura Frontend SPA y Móvil

- **SPA Web:** Se usará los frameworks Angular o React.
Angular si se busca estructura rígida y control empresarial.
React si se prioriza flexibilidad y velocidad de desarrollo.

Criterio	Justificación
Ecosistema maduro	Ambos frameworks cuentan con amplio soporte empresarial, documentación robusta y comunidades activas. Angular es respaldado por Google y React por Meta, lo que garantiza continuidad y evolución.
Integración con OAuth2.0 y API REST	Angular y React permiten integrar fácilmente flujos de autenticación como Authorization Code Flow con PKCE, usando bibliotecas como angular-oauth2-oidc o react-oauth2. Además, su arquitectura basada en componentes facilita el consumo de APIs RESTful.
Manejo de estados y rutas	Angular ofrece un sistema de rutas integrado y gestión de estados con NgRx. React permite usar React Router y bibliotecas como Redux o Zustand para manejar estados complejos. Esto es clave para flujos como transferencias, pagos y consultas.
Escalabilidad y mantenibilidad	Angular promueve una arquitectura modular con servicios inyectables, ideal para proyectos bancarios. React, con hooks y componentes reutilizables, permite construir interfaces dinámicas y desacopladas.

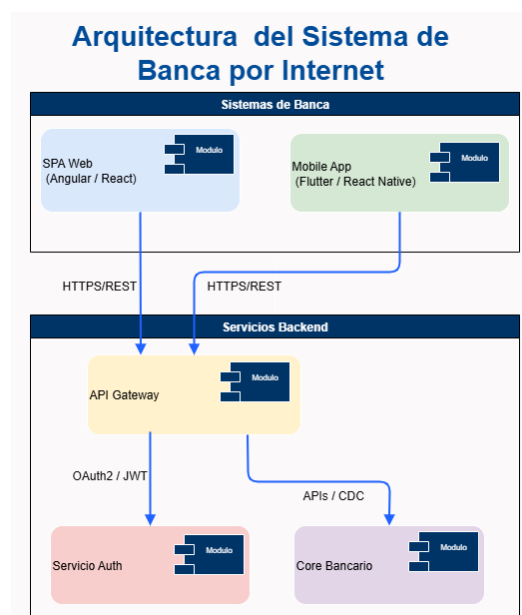
Seguridad y cumplimiento	Ambos permiten implementar CSP, sanitización de inputs, y protección contra XSS/CSRF, alineándose con normativas como OWASP y PCI-DSS.
--------------------------	--

- **App Móvil:** Flutter o React Native.

Flutter si se prioriza rendimiento gráfico y control visual.

React Native si se busca integración rápida con SDKs existentes y ecosistema JS compartido con el SPA.

Criterio	Justificación
Multiplataforma (iOS/Android)	Ambos frameworks permiten desarrollar una sola base de código para múltiples plataformas, reduciendo costos y tiempos de entrega.
Integración con biometría y SDKs nativos	Flutter y React Native permiten integrar biometría (huella, rostro) mediante plugins como local_auth (Flutter) o react-native-fingerprint-scanner. También permiten integrar SDKs bancarios, notificaciones push y cámaras para onboarding facial.
Rendimiento y experiencia de usuario	Flutter compila a código nativo, ofreciendo alto rendimiento gráfico. React Native usa el puente nativo, con buen rendimiento y acceso a componentes nativos. Ambos permiten animaciones fluidas y UX moderna.
Comunidad y soporte	Flutter tiene fuerte respaldo de Google y una comunidad creciente. React Native tiene una comunidad consolidada y gran cantidad de librerías disponibles.
Seguridad y cumplimiento	Permiten cifrado local, almacenamiento seguro (Secure Storage / Keychain), y autenticación biométrica, alineándose con estándares como ISO/IEC 27001 y normativas locales.



Integración con Backend y Seguridad

- Ambas soluciones consumen servicios vía API Gateway con tokens OAuth2.0.
- Se recomienda usar **Authorization Code Flow con PKCE** para evitar exposición de credenciales en clientes públicos.
- Se puede implementar **SSO** si la entidad bancaria lo requiere, usando Azure AD B2C o Auth0.

Valor estratégico

- Reducción de costos operativos por reutilización de código.
- Experiencia de usuario consistente en todos los canales.
- Flexibilidad para incorporar nuevas funcionalidades como onboarding facial, notificaciones, o pagos QR.
- Cumplimiento normativo y alineación con estándares bancarios.

7. Arquitectura de Autenticación y Autorización

Flujo de Autenticación: OAuth2.0 con Authorization Code Flow + PKCE

Justificación:

- **OAuth2.0 + PKCE:**
 - ✓ Es el estándar recomendado por la IETF para aplicaciones móviles y SPA.
 - ✓ Reduce riesgos de ataques *man-in-the-middle* y *token replay*.
- **Authorization Code Flow con PKCE:** es el flujo recomendado para clientes públicos como SPA y apps móviles, ya que:
 - ✓ No requiere almacenamiento de secretos en el cliente.
 - ✓ Protege contra ataques de interceptación de tokens.
 - ✓ Compatible con navegadores y dispositivos móviles.

Seguridad y compatibilidad

- **PKCE (Proof Key for Code Exchange):** agrega una capa de seguridad adicional al flujo estándar.

Compatible con proveedores como:

- ✓ IdentityServer (auto-hosted, extensible).
- ✓ Auth0 (SaaS, rápido de implementar).
- ✓ Azure AD B2C (integración con ecosistemas Microsoft, escalabilidad empresarial).

Flujo resumido:

1. Cliente inicia login → redirige a proveedor OAuth2.0.

2. Usuario se autentica → se genera código de autorización.
3. Cliente envía código + PKCE verifier → recibe token de acceso.
4. Token se usa para consumir APIs protegidas.

- **Framework multiplataforma (Flutter/React Native):**
 - ✓ Reducción de costos y tiempos de desarrollo con una sola base de código.
 - ✓ Ecosistema maduro con librerías de integración biométrica y soporte empresarial.
- **Servicios biométricos (Azure Face API / Amazon Rekognition):**
 - ✓ Escalabilidad y precisión con algoritmos entrenados en millones de rostros.
 - ✓ Cumplimiento normativo internacional (ISO/IEC 30107, FIDO2, GDPR, LOPDP).

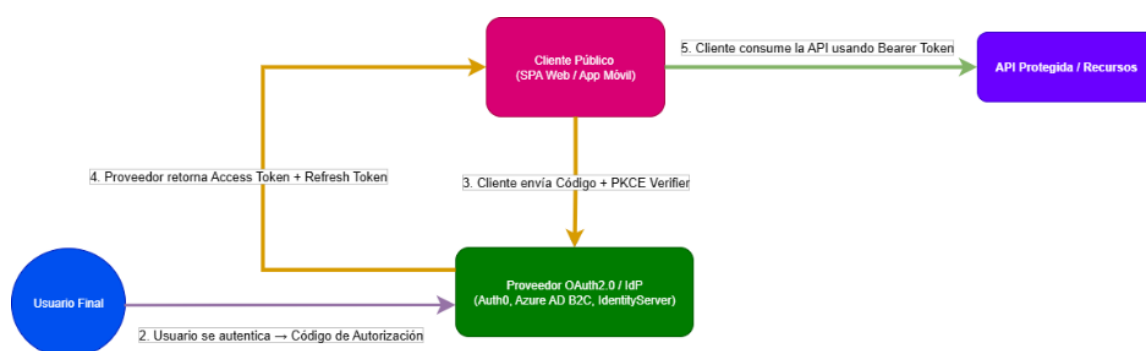


Ilustración 7: Flujo de Autenticación.

Onboarding Biométrico con Reconocimiento Facial

Mejora la experiencia de usuario y reduce fricción en el registro.

- ✓ Fortalece la seguridad desde el primer contacto.
- ✓ Cumple con estándares de autenticación biométrica (FIDO2, ISO/IEC 30107).

Herramienta	Características
Azure Face API	Detección facial, verificación, reconocimiento. Integración con Azure AD B2C.
Amazon Rekognition	Análisis facial, comparación de rostros, integración con Cognito y Lambda.

Flujo de Onboarding

1. Usuario instala app móvil y accede al flujo de registro.
2. Captura facial → se valida contra base de datos o patrón biométrico.
3. Si es exitoso → se generan credenciales (usuario/clave).
4. Se habilita autenticación futura por:

- Usuario + clave
- Huella digital (Android Biometric API)
- Reconocimiento facial (iOS FaceID / Android Face Unlock).

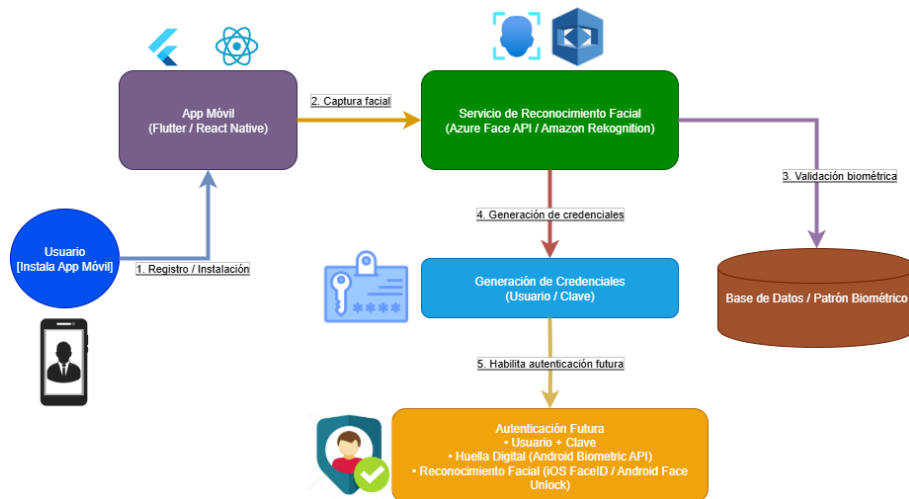


Ilustración 8: Flujo de Reconocimiento Facial BP

Justificación del flujo de reconocimiento Facial:

El flujo de reconocimiento facial constituye el punto de partida del proceso de onboarding digital dentro del sistema de banca por internet. Su objetivo es garantizar una experiencia de registro segura, personalizada y alineada con los estándares internacionales de autenticación biométrica.

Instalación y Registro del Usuario: Durante la instalación y registro inicial, el usuario descarga la aplicación móvil y crea su perfil, estableciendo las bases para una experiencia personalizada y segura. Este paso representa el **punto de entrada al ecosistema de autenticación biométrica**, donde se vincula la identidad digital del cliente con su dispositivo.

Frameworks Multiplataforma: El uso de Flutter o React Native permite desarrollar una única base de código para iOS y Android, reduciendo costos y tiempos de desarrollo. Además, garantiza uniformidad en la experiencia de usuario y facilita la integración con librerías de captura facial y servicios biométricos cloud.

Captura y Validación Biométrica: La captura facial es el primer paso para asociar la identidad biométrica del usuario con su cuenta bancaria. La validación biométrica se realiza mediante Azure Face API o Amazon Rekognition, servicios que ofrecen algoritmos avanzados de reconocimiento y verificación facial. Delegar esta tarea a servicios cloud garantiza:

- ✓ **Escalabilidad** ante grandes volúmenes de usuarios.
- ✓ **Precisión y confiabilidad** en la comparación de patrones faciales.

- ✓ **Cumplimiento normativo** con estándares internacionales (ISO/IEC 30107, FIDO2, GDPR, LOPDP).

Generación de Credenciales Tradicionales: Aunque el sistema utiliza autenticación biométrica, se generan credenciales tradicionales (usuario/clave) como respaldo. Esto asegura compatibilidad con otros sistemas y provee una capa adicional de seguridad ante fallos biométricos o accesos desde dispositivos no compatibles.

Almacenamiento del Patrón Biométrico: El patrón facial cifrado se almacena en la base de datos siguiendo políticas de seguridad y cifrado con KMS (Key Management Service). Este almacenamiento permite futuras comparaciones para autenticación, manteniendo la persistencia y trazabilidad de la identidad del usuario.

Autenticación Futura: Una vez completado el registro, el usuario puede autenticarse en futuras sesiones mediante reconocimiento facial sin repetir el proceso inicial. Esto mejora la experiencia de usuario, reduce fricción en el acceso y fortalece la seguridad adaptativa del sistema.

Usuario + Clave: Método tradicional como respaldo.

Huella Digital (Android Biometric API): Aprovecha sensores nativos para autenticación rápida y segura.

Reconocimiento Facial (iOS FaceID / Android Face Unlock): Utiliza tecnologías nativas del sistema operativo para una experiencia fluida y segura.

Beneficios Generales del Flujo:

Seguridad: Combina biometría con credenciales tradicionales.

- ✓ **Escalabilidad:** Uso de servicios cloud como Azure y AWS.
- ✓ **Compatibilidad:** Funciona en múltiples plataformas y dispositivos.
- ✓ **Experiencia de Usuario:** Flujo intuitivo y rápido para el usuario final.

Integración con arquitectura

- El **servicio de onboarding** se desacopla como microservicio independiente.
- Se comunica con el **servicio de autenticación OAuth2.0** para registrar nuevos usuarios.
- Se integra con el **servicio de auditoría** para registrar eventos críticos (registro, autenticación, fallos).
- Se conecta con el **servicio de notificaciones** para confirmar al usuario su registro exitoso.

8. Arquitectura de Integración

API Gateway: El API Gateway constituye el punto central de entrada al ecosistema de servicios:

- ✓ **Funciones principales:** autenticación (OAuth2.0 + PKCE), autorización, control de tráfico, versionamiento y monitoreo.
- ✓ **Beneficios estratégicos:**
 - Centraliza la seguridad y el acceso.
 - Reduce la exposición directa de endpoints internos.
 - Facilita la gobernanza y el ciclo de vida de las APIs.

Servicios Expuestos: El sistema expone inicialmente tres servicios clave, alineados con los dominios BIAN:

1. **Datos Básicos:** consultas de información del cliente (perfil, saldos, datos generales).
2. **Movimientos:** historial de transacciones, implementado con **Event Sourcing** para trazabilidad.
3. **Transferencias:** operaciones críticas internas e interbancarias, gestionadas con el **Saga Pattern** para consistencia eventual y resiliencia.

Cada servicio se publica a través del API Gateway con documentación estandarizada (**OpenAPI/Swagger**) y soporte para mensajería asíncrona (**AsyncAPI**).

Estrategia de Expansión Futura: La arquitectura está diseñada para crecer de manera modular y escalable:

- **Nuevos servicios:** incorporación de pagos digitales, onboarding avanzado, prevención de fraudes y servicios regulatorios.
- **Escalabilidad horizontal:** despliegue de microservicios adicionales sin afectar los existentes.
- **Integración con terceros:** apertura hacia fintechs y reguladores mediante APIs externas, cumpliendo estándares de **Open Finance**.
- **Observabilidad y gobierno:** integración continua con **API Manager**, dashboards de consumo y auditoría en tiempo real.
- **Resiliencia y HA/DR:** soporte para clusters distribuidos y recuperación ante desastres, garantizando continuidad operativa.

9. Persistencia y Auditoría

Patrones de Diseño (CQRS + Event Sourcing)

La arquitectura de persistencia aplica los patrones **CQRS (Command Query Responsibility Segregation)** y **Event Sourcing**:

- ✓ **CQRS:** separa las operaciones de lectura y escritura, optimizando rendimiento y escalabilidad.

- *Write Side*: Aurora PostgreSQL para transacciones críticas.
 - *Read Side*: DynamoDB/Elasticsearch para consultas rápidas y reportes.
 - Mejora el rendimiento al desacoplar cargas de lectura y escritura.
 - Facilita la escalabilidad horizontal y la resiliencia en entornos distribuidos.
- ✓ **Event Sourcing**: cada acción del cliente (ej. transferencia, pago) se registra como un evento inmutable.
- Permite reconstruir el estado del sistema en cualquier momento.
 - Garantiza trazabilidad y cumplimiento normativo (ISO 27001, PCI DSS).
 - Proporciona auditoría completa y verificable.
 - Simplifica la recuperación ante desastres al poder regenerar el estado desde eventos.

Registro de Acciones y Almacenamiento para Clientes Frecuentes

El sistema implementa un **mecanismo de logging y persistencia** que:

- ✓ Registra todas las acciones del cliente en el **Event Store** con correlación única (ID de transacción).
- ✓ Mantiene un **historial completo de transacciones** para auditoría y análisis.
- ✓ Utiliza **persistencia optimizada para clientes frecuentes**, almacenando patrones de uso y preferencias en cache (Redis) para reducir latencia y mejorar la experiencia de usuario.
- ✓ Facilita la generación de reportes regulatorios y dashboards de consumo en tiempo real.
- ✓ Patrones aplicados:
 - CQRS: separación de consultas frecuentes vs. escrituras críticas.
 - Event Sourcing: trazabilidad total de eventos.
 - Log Compaction (Kafka): optimización de almacenamiento histórico.

Notificaciones: La capa de auditoría se integra con el Sistema de Notificaciones para garantizar comunicación redundante:

- ✓ **Eventos críticos** (transferencias, pagos, accesos sospechosos) generan notificaciones inmediatas vía SMS, Push o Email.
- ✓ Se utiliza el **Outbox Pattern** para asegurar que ningún mensaje se pierda en caso de fallo del broker.
- ✓ Integración con **Twilio, Firebase Push y AWS SES/SNS** para entrega confiable y multicanal.
- ✓ Las notificaciones también se registran en el **Event Store**, reforzando trazabilidad y cumplimiento.
- ✓ Asegura consistencia entre eventos de negocio y notificaciones enviadas.

- ✓ Cumple con normativas de comunicación financiera (ISO 22301, continuidad del negocio).

10. Notificaciones

Integración con al menos dos sistemas: El sistema de notificaciones se integra con proveedores externos para garantizar entrega confiable y multicanal:

- ✓ **Twilio SMS / AWS SNS:** envío de mensajes de texto para alertas críticas (transferencias, accesos sospechosos).
- ✓ **Firebase Push / AWS SNS Push:** notificaciones en tiempo real hacia dispositivos móviles (Android/iOS).
- ✓ **SendGrid / AWS SES:** envío de correos electrónicos para confirmaciones y reportes.

Esta integración permite flexibilidad en la elección de proveedores y asegura continuidad operativa incluso si uno de ellos falla.

Estrategia de Redundancia: La arquitectura aplica una **estrategia de redundancia activa**:

- ✓ Cada evento crítico (ej. transferencia, pago, intento de acceso) se envía a **dos canales distintos** (ej. SMS + Push).
- ✓ Se implementa el **Outbox Pattern** en los microservicios, garantizando que los mensajes se registren primero en la base de datos antes de enviarse al broker.
- ✓ El **Bus de Mensajes (Kafka / RabbitMQ / SQS)** asegura que las notificaciones se procesen de forma asíncrona y tolerante a fallos.
- ✓ Los adaptadores externos (Twilio, Firebase, SES) están desacoplados, permitiendo sustituir proveedores sin afectar la lógica interna.

Entrega Garantizada: Para asegurar la entrega de notificaciones:

- ✓ Se implementan **reintentos automáticos** en caso de fallo de envío.
- ✓ Se utiliza una **Dead Letter Queue (DLQ)** para almacenar mensajes no entregados y reprocesarlos posteriormente.
- ✓ El sistema registra cada notificación en el **Event Store**, garantizando trazabilidad y auditoría.
- ✓ Se aplican métricas de observabilidad (Prometheus/Grafana) para monitorear tasas de entrega, latencia y errores.

11. Infraestructura en la nube

Opciones: AWS vs Azure

La arquitectura puede desplegarse en **AWS** o **Azure**, ambos con servicios nativos que soportan microservicios, integración y seguridad:

- ✓ **AWS:** ofrece servicios maduros como **Aurora PostgreSQL, DynamoDB, SQS, SNS, Rekognition**, con fuerte presencia en Latinoamérica y amplia comunidad de soporte.
- ✓ **Azure:** integra de forma nativa con **Active Directory, Face API, Service Bus, Event Grid, AKS**, lo que facilita la gobernanza y el cumplimiento regulatorio en entornos corporativos.

Justificación de Elección: La decisión se basa en tres factores estratégicos:

- ✓ **Latencia:** AWS dispone de regiones en Sudamérica (ej. São Paulo), lo que reduce la latencia para usuarios en Ecuador y Chile. Azure también ofrece baja latencia con su red global, pero con menos presencia directa en la región.
- ✓ **Costos:** AWS suele ofrecer mayor flexibilidad en modelos de precios por consumo, mientras que Azure resulta competitivo en entornos corporativos con licencias Microsoft ya existentes.
- ✓ **Servicios Nativos:**
 - AWS destaca en **event streaming, bases NoSQL y biometría (Rekognition)**.
 - Azure sobresale en **integración empresarial, seguridad y biometría (Face API)**.

En este caso, se recomienda **AWS como plataforma principal** por su latencia más baja en la región y su robustez en servicios nativos para microservicios y biometría, complementado con **Azure para integración corporativa y autenticación**.

Alta Disponibilidad, Tolerancia a Fallos y Recuperación ante Desastres

La infraestructura se diseña bajo principios de **resiliencia y continuidad operativa**:

- **Alta Disponibilidad (HA):** despliegue en **Multi-AZ** con balanceadores de carga y autoescalado.
- **Tolerancia a Fallos:** uso de **event streaming (Kafka/MSK)** y colas distribuidas (**SQS/Service Bus**) para garantizar consistencia eventual.
- **Recuperación ante Desastres (DRP):**
 - Replicación de datos en múltiples regiones.
 - Backups automáticos cifrados en S3/Blob Storage.
 - Failover gestionado con **Route53 (AWS)** o **Traffic Manager (Azure)**.

- Simulaciones periódicas de fallos para validar RPO/RTO definidos.

12. Seguridad y cumplimiento Normativos

En la arquitectura propuesta de solución de la integración Bancaria, se considera el cumplimiento de las siguientes normativas y estándares relevantes para el sector financiero:

- Ley Orgánica de Protección de Datos Personales (LOPDP – Ecuador)
- Reglamento General de Protección de Datos (GDPR – si aplica por jurisdicción de usuarios)
- PCI DSS (Payment Card Industry Data Security Standard)
- ISO/IEC 27001: Gestión de Seguridad de la Información
- ISO 20022 : Es un estándar internacional de mensajería financiera que define un lenguaje común para las transacciones electrónicas.
- ISO 22301: Continuidad del Negocio y Recuperación ante Desastres
- OWASP ASVS (Application Security Verification Standard)
- Ley de Servicios Financieros y normativa de la Superintendencia de Bancos (Ecuador).

Justificación de Cumplimiento

La arquitectura en AWS garantiza el cumplimiento normativo mediante los siguientes mecanismos:

Requisito Normativo	Mecanismo de Cumplimiento en la Arquitectura
Protección de datos personales	Uso de AWS Cognito para autenticación segura, cifrado en tránsito (TLS 1.3) y en reposo ((AES-256 con KMS), segmentación por VPCs.
Trazabilidad y auditoría	Implementación de CloudTrail y CloudWatch Logs para registrar acciones, accesos y eventos críticos. ; Event Sourcing para reconstrucción de estado.
Seguridad de aplicaciones	Integración de AWS WAF, Shield, y validaciones OWASP en el backend (API Gateway + Lambda).
Alta disponibilidad y recuperación ante desastres (DR)	Arquitectura distribuida en múltiples zonas de disponibilidad, uso de Amazon S3 y RDS Multi-AZ, replicación multi-región.
Autenticación robusta	Uso de OAuth2.0 con Cognito, flujos PKCE para móviles, integración con biometría (Azure Face API, Amazon Rekognition).
Monitoreo y respuesta ante incidentes	Activación de GuardDuty, Config, y alertas en CloudWatch para detección temprana de amenazas ; Prometheus + Grafana para métricas avanzadas.
Segregación de ambientes y funciones	Separación por VPCs: Frontend, Backend, Seguridad, Datos, Core Bancario, con roles IAM específicos y políticas granulares.

Riesgos Mitigados

La arquitectura propuesta mitiga los siguientes riesgos críticos para una entidad financiera:

Riesgo	Mitigación
Acceso no autorizado a datos sensibles	IAM con políticas granulares, autenticación multifactor, cifrado de datos.
Ataques DDoS o inyecciones web	Protección con AWS Shield, WAF, validaciones en API Gateway.
Pérdida de trazabilidad o auditoría	Registro completo de eventos con CloudTrail, retención en S3.
Fallas en disponibilidad del servicio	Arquitectura HA con balanceo de carga, replicación en RDS, auto-healing.
Incumplimiento normativo	Diseño alineado a estándares internacionales y locales, documentación técnica y legal.
Exposición de credenciales o tokens	Uso de flujos seguros OAuth2.0, almacenamiento cifrado, rotación de secretos.

13. Gobernanza de APIs y Microservicios

Modelo de Gobierno: El modelo de gobierno asegura que la arquitectura de APIs y microservicios se mantenga **ordenada, segura, escalable y trazable** en el tiempo:

- ✓ **API Manager (Apigee / AWS API Gateway / Azure API Management):** catálogo centralizado, versionado, políticas de seguridad y auditoría de consumo.
- ✓ **Service Mesh (Istio / Linkerd):** control de tráfico interno, seguridad en la comunicación entre microservicios, trazabilidad y resiliencia.
- ✓ **CI/CD Pipeline (Jenkins / GitHub Actions / Azure DevOps):** automatización de despliegues, validación de contratos y pruebas de regresión.
- ✓ **Observabilidad (Prometheus / Grafana / ELK / OpenTelemetry):** métricas, trazas y alertas en tiempo real para garantizar SLA y cumplimiento normativo.

Este modelo permite que cada microservicio tenga responsabilidades claras, con políticas de acceso y monitoreo unificados.

Estrategia de Versionamiento: El versionamiento de APIs y microservicios es clave para garantizar estabilidad y evolución controlada:

- ✓ **Semantic Versioning (SemVer):** uso de versiones mayores, menores y parches (ej. v1.0.0 → v1.1.0 → v2.0.0).
- ✓ **Versionado en URLs o Headers:** exposición de múltiples versiones activas para compatibilidad con clientes externos.

- ✓ **Backward Compatibility:** mantenimiento de versiones anteriores mientras se migran consumidores a nuevas versiones.
- ✓ **Ciclo de Vida de APIs:** diseño → publicación → consumo → monitoreo → retiro, gestionado desde el API Manager.
- ✓ **Auditoría de Consumo:** registro de qué clientes consumen qué versión, con alertas ante uso de versiones obsoletas.

Estrategia de Seguridad: La seguridad se implementa de forma transversal en APIs y microservicios:

- ✓ **Autenticación y Autorización:** OAuth2.0 + PKCE, JWT con rotación de claves, integración con IAM corporativo.
- ✓ **Políticas de Acceso Dinámicas:** control por roles y scopes, aplicadas desde el API Gateway y Service Mesh.
- ✓ **Rate Limiting y Throttling:** protección contra abusos y ataques de denegación de servicio.
- ✓ **Cifrado en tránsito y reposo:** TLS 1.3 para comunicaciones, KMS para gestión de claves y secretos.
- ✓ **Cumplimiento Normativo:** alineación con PCI DSS, ISO 27001 y Habeas Data.

14. Patrones de integración y tecnologías

Presentamos los patrones de diseño para una Arquitectura desacoplada, reusable y cohesionada.

Patrón	Justificación
Microservicios	Permite escalar servicios de manera independiente en este caso las transferencias, pagos, consultas y facilita el despliegue continuo (CI/CD) y reduce riesgos de fallos globales.
CQRS (Command Query Responsibility Segregation)	Optimiza consultas de historial y operaciones de escritura separando cargas y mejora la trazabilidad y auditoría, clave en entornos financieros regulados.
Event Sourcing	Registra cada acción del cliente como evento inmutable, garantizando trazabilidad que proporciona reconstrucción del estado del sistema en caso de fallos o auditorías.
Saga Pattern (Orquestación de transacciones distribuidas)	Maneja transferencias interbancarias con consistencia eventual y compensaciones, y reduce riesgos de inconsistencias en operaciones críticas multi-servicio.
Adapter Pattern	Aísla la lógica de negocio de sistemas externos (Core Tradicional y Complementario), que permite cambiar proveedores sin afectar el núcleo del sistema.
Circuit Breaker	Protege contra fallos en servicios externos, evitando cascadas de errores que mejora la resiliencia y la experiencia del cliente en caso de indisponibilidad parcial.

- A. **Event-driven:** Esta arquitectura está orientada a eventos con un estilo en el que el sistema core bancario digital se comunica mediante eventos distribuidos en lugar de llamadas síncronas, el cual se hará uso de Kafka/MSK, donde quedan desacoplados los microservicios a usarse.
El Beneficio de este patrón es baja latencia, resiliencia, e integración con terceros (Open Finance)
- B. **CQRS + Event Sourcing :** Permite la separación de lectura/escritura y trazabilidad.

Command Query Responsibility Segregation separa las operaciones de lectura (queries) de las de escritura (commands).

- En lugar de tener un único modelo para todo, se usan dos modelos especializados:

- ✓ Command Model: para modificar el estado.
- ✓ Query Model: para consultar el estado.

El propósito del Event Sourcing es:

- En lugar de guardar el estado actual, se guarda una secuencia de eventos que llevaron a ese estado.
- Cada acción (ej. transferencia realizada) se registra como un evento inmutable.

Aplicación en el sistema bancario digital contempla;

- ✓ **Transferencias:** cada vez que un usuario realiza una transferencia, se genera un evento.
- ✓ **Historial de transacciones:** se reconstruye desde los eventos, garantizando trazabilidad completa.
- ✓ **Auditoría:** los eventos son perfectos para cumplir con normativas como ISO 27001 o PCI-DSS.

Beneficios:

- ✓ Trazabilidad total.
- ✓ Escalabilidad: puedes optimizar las consultas sin afectar la lógica de negocio.
- ✓ Integración con MSK para persistencia de eventos.

- C. **Saga Pattern:** Este patrón permite manejar transacciones distribuidas en microservicios, como por ejemplo para orquestación de transferencias interbancarias.
- Cada paso de la transacción es un servicio independiente que emite eventos y escucha respuestas.
 - Si algo falla, se ejecutan acciones compensatorias.
- Desacopla la lógica de negocio de proveedores externos(SMS, Push, API Manager).

Ejemplo:

Aplicación en transferencias interbancarias

1. Servicio A inicia transferencia → evento "Transferencialniciada"
2. Servicio B debita cuenta origen → evento "CuentaDebitada"
3. Servicio C acredita cuenta destino → evento "CuentaAcreditada"
4. Si falla el paso 3 → se emite "ReversarDebito"

Beneficios

- Evita bloqueos de recursos.
- Alta disponibilidad y resiliencia.
- Compatible con arquitecturas basadas en eventos (Kafka, RabbitMQ).

- D. Adapter Pattern:** Este patrón permite desacoplar servicios de notificación y que el sistema interactúe con múltiples servicios externos sin depender directamente de ellos, se crea una interfaz común y cada servicio externo tiene su propio adaptador.

Beneficios

- Puedes cambiar de proveedor sin modificar la lógica de negocio.
- Facilita pruebas unitarias y simulaciones.
- Mejora la mantenibilidad y extensibilidad.

- E. Factory Pattern :** Este patrón es para creación de clientes en onboarding y encapsula la lógica de creación de objetos en una clase especializada.

- Ideal cuando el proceso de creación depende de múltiples condiciones o configuraciones.

Beneficios:

- Centraliza la lógica de creación.
- Permite validar y enriquecer datos antes de instanciar el objeto.
- Facilita la incorporación de nuevos tipos de clientes (ej. corporativos, VIP).

- F. Orquestación multicore con microservicio:** Este patrón buscamos la independencia del Core Bancario Tradicional y el Core Bancario Digital, mediante la tecnología de Kubernetes y Services Mesh (Istio)

Beneficio:

Service Mesh con mTLS, observabilidad, balanceo de carga y tolerancia a fallo.

Integración en la Arquitectura del Sistema de Banca por Internet

Patrón	Ubicación en C4	Propósito
Event driven	Backend eventos distribuidos	Des acoplamiento entre microservicios.
CQRS + Event Sourcing	Backend API + Auditoría	Separación de responsabilidades y trazabilidad.
Saga Pattern	Servicio de Transferencias	Orquestación distribuida.
Adapter Pattern	Servicio de Notificaciones	Desacoplamiento de proveedores.

Factory Pattern	Servicio de Onboarding	Creación flexible de clientes.
Orquestación multicore	En los Contenedores	Independencia de core tradicional y core banca digital

15. Infraestructura en la nube (AWS)

Se tiene la siguiente matriz donde se detalla la justificación técnica y estratégica:

Requisito	Solución	Justificación
Alta disponibilidad (HA)	AWS ECS Fargate con múltiples zonas	Permiten desplegar contenedores o aplicaciones web en múltiples zonas de disponibilidad. Esto garantiza que si una zona falla, el tráfico se redirige automáticamente a otra. Ideal para microservicios desacoplados.
Tolerancia a fallos	AWS Lambda Auto Scaling + Load Balancer	Lambda es Auto Scaling ajusta la cantidad de instancias según la carga. El Load Balancer distribuye el tráfico entre instancias saludables. Esto evita caídas por sobrecarga y asegura continuidad operativa.
Recuperación ante desastres (DR)	Backups en S3 + replicación geográfica	S3 permiten backups automáticos y replicación entre regiones. Esto protege contra pérdida de datos por fallos físicos o ciberataques. Se puede configurar recuperación punto en el tiempo.
Seguridad	IAM Roles, AWS Secrets Manager, WAF, TLS, AWS Shield Advanced	IAM controla acceso granular a recursos. Secrets Manager protegen credenciales y claves. WAF (Web Application Firewall) bloquea ataques como XSS y SQLi. TLS cifra la comunicación. AWS Shield Advanced es un servicio administrado de protección contra ataques DDoS (Denegación de Servicio Distribuido) Todo esto cumple con PCI-DSS, ISO 27001 y normativas locales.
Monitoreo	CloudWatch + Grafana	CloudWatch recolectan métricas, logs y alertas. Grafana permite visualización avanzada y dashboards personalizados. Esto facilita la observabilidad, análisis de rendimiento y respuesta proactiva.
Auto-healing	Health checks + reinicio automático	Las instancias se monitorean constantemente. Si una falla, se reinicia automáticamente o se reemplaza. Esto reduce el tiempo de inactividad y mejora la resiliencia del sistema.

Arquitectura recomendada en la nube de AWS

- **Frontend SPA:** Deploy en S3 + CloudFront (CDN).
 - ✓ **S3 + CloudFront:** Hosting estático + CDN para baja latencia global.
 - ✓ **WAF + Shield:** Protección contra ataques DDoS y amenazas web.
 - ✓ **Route 53:** DNS resiliente y balanceo geográfico si se requiere.
- **Seguridad:** IAM, Secrets Manager, WAF, Shield, GuardDuty.
 - ✓ **IAM:** Control de acceso granular.
 - ✓ **Secrets Manager:** Gestión segura de credenciales.
 - ✓ **GuardDuty:** Detección de amenazas.
 - ✓ **AWS Config + Security Hub:** Cumplimiento normativo continuo.

- **App móvil:** Backend en ECS Fargate + API Gateway.
- **ECS Fargate:** Ejecuta la lógica de negocio de la aplicación móvil , en este caso validaciones, procesamiento de pagos, reglas de negocio, también, expone endpoints REST que la app móvil consume y se comunica con servicios como bases de datos, sistemas de auditoría, APIs internas.

Permite escalar automáticamente según la carga (más usuarios, más contenedores, y aísla cada microservicio como las transferencias, pagos, e historial.

- **API Gateway + Lamda (Serverless):** este va a contemplar los diferentes API que se mencionan a continuación desacoplados cada uno con Lamda .
 - ✓ Servicio API de Clientes.
 - ✓ Servicio API de Movimientos.
 - ✓ Servicio API de Transferencias.
 - ✓ Servicio API de Transferencias.
 - ✓ Servicio API de Pagos.

Integración con **Amazon SNS** y **Amazon Pinpoint** para SMS, email, push.

- **Base de datos:** RDS PostgreSQL con replicación multi-AZ.
 - ✓ **RDS PostgreSQL Multi-AZ:** Transaccional, alta disponibilidad.
 - ✓ **Amazon DynamoDB (MongoDB compatible):** Auditoría flexible.
- **Auditoría:** DynamoDB + eventos en Kafka (MSK).
- **Monitoreo:** CloudWatch + Grafana + SNS para alertas.

Valor estratégico:

1. **Escalabilidad automática:** Se adapta a la demanda sin intervención manual.
2. **Resiliencia operativa:** Minimiza el impacto de fallos y ataques.
3. **Cumplimiento normativo:** Compatible con estándares bancarios y regulatorios.
4. **Costos controlados:** Pago por uso, con posibilidad de optimización por horarios o tráfico.
5. **Desacoplamiento total:** Cada componente puede evolucionar sin afectar al resto.

La arquitectura está diseñada bajo principios de modularidad, seguridad, escalabilidad y trazabilidad, alineada con estándares internacionales y buenas prácticas de AWS. Cada VPC representa un dominio funcional independiente, lo que permite aislamiento lógico, control granular de accesos y facilidad de mantenimiento.

Relación entre Componentes de la Arquitectura de nube AWS

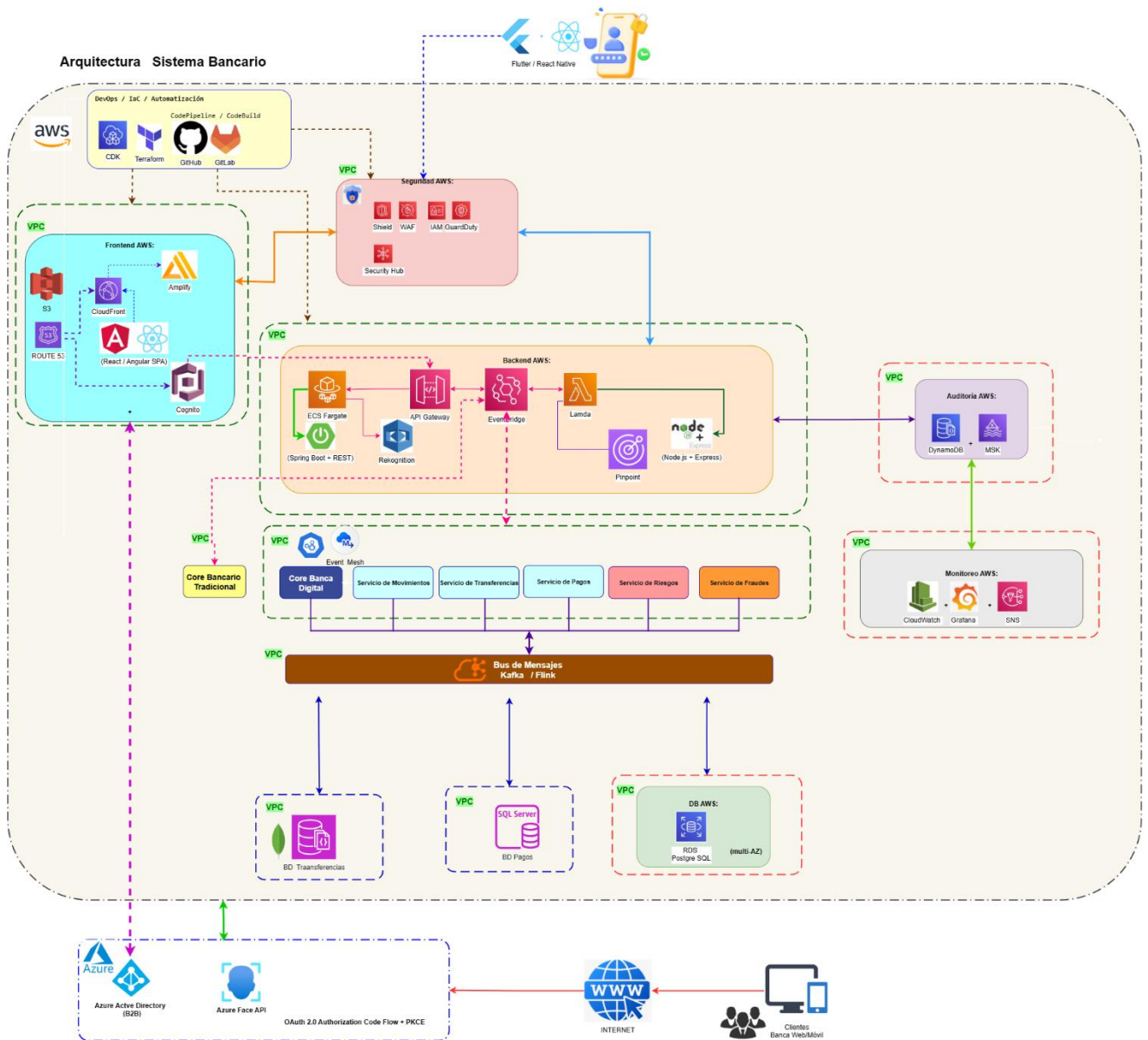


Ilustración 9:Arquitectura Bancaria en AWS.

VPC / Dominio	Propósito Funcional	Herramientas / Servicios Clave	Buenas Prácticas
Frontend AWS	Acceso público, entrega de contenido, autenticación y hosting web	Route 53, CloudFront, WAF, Cognito, S3, Amplify, API Gateway	- Usar WAF para protección contra OWASP Top 10 - Cognito para federación segura - Amplify para CI/CD frontend
Seguridad AWS	Monitoreo, gestión de identidades y cumplimiento normativo	GuardDuty, Security Hub, IAM, Config	- Activar GuardDuty y Security Hub para detección proactiva - Config + CloudTrail para auditoría continua
Backend AWS	Lógica de negocio, persistencia de datos y configuración centralizada	EC2, RDS, ElastiCache, S3, AppConfig,	- Usar AppConfig para despliegues seguros - Parameter Store cifrado para secretos - RDS con backups automáticos
Core Bancario Digital	Procesamiento de operaciones modernas (pagos, transferencias, movimientos)	Microservicios (Spring Boot / .NET), Kafka / Flink, PostgreSQL, Redis	- Arquitectura desacoplada y resiliente - Cumplimiento ISO 20022 y BIAN - Kafka para integración por eventos
Core Bancario Tradicional	Procesamiento de datos legacy y productos financieros existentes	IBM ACE, Apache Camel, SQL Server, SOAP	- Adaptadores para coexistencia multicore - Monitoreo de integraciones legacy - Migración gradual con mínima interrupción
Plataforma de Integración	Comunicación entre cores y servicios externos	Kafka, Event Mesh, SAGA, CQRS, Event Sourcing	- Garantizar consistencia eventual - Implementar patrones de resiliencia y compensación - Monitoreo de flujos de eventos
Gestión de Riesgos y Fraudes	Evaluación crediticia y detección de anomalías	ML + CEP, AWS SageMaker, motores analíticos	- Modelos ML entrenados con datos auditables - CEP para detección en tiempo real
Serverless AWS	Procesamiento sin servidor, eventos y mensajería desacoplada	Lambda, EventBridge, SQS, SNS, DynamoDB	- Lambda con IAM mínimo necesario - SQS/SNS para resiliencia
Data & Control	Observabilidad, trazabilidad y automatización CI/CD	CloudWatch, CloudFormation, CodePipeline, CodeBuild, CodeDeploy	- CloudWatch con alarmas por VPC-CodePipeline con validaciones

Auditoría y Monitoreo	Registro y observabilidad del sistema	DynamoDB, MSK, CloudWatch, Grafana, ELK	- Centralizar logs y métricas - Alertas automáticas ante fallos - Auditoría continua para cumplimiento normativo
Notificaciones y Comunicación	Alertas críticas y confirmaciones al cliente	AWS SNS, Pinpoint, SMS Gateway	- Comunicación multicanal - Mensajería desacoplada - Confirmaciones seguras y trazables
Infraestructura / HA / DRP	Alta disponibilidad, resiliencia y recuperación ante desastres	Kubernetes (EKS/AKS), RDS Multi-AZ, Backup cross-region, Terraform / CDK	- Clusters activos-activos - Replicación cross-region - Infraestructura como código (IaC)
Integraciones Externas	Identidad federada, control de versiones y conectividad externa	Azure AD, Azure DevOps, GitHub, Internet	- Azure AD para SSO empresarial - GitHub con escaneo SAST/SCA - Control de egress con NAT/WAF
One Branch	Punto de control o acceso unificado entre dominios	Conexión entre VPCs y servicios	- Usar Transit Gateway o VPC Peering - Control de tráfico con Security Groups y NACLs

La solución está diseñada para garantizar alta disponibilidad, seguridad y trazabilidad en un entorno bancario. Cada componente cumple una función específica dentro de un ecosistema desacoplado y monitoreado, donde la autenticación, la lógica de negocio y la persistencia de datos están aisladas pero interconectadas estratégicamente. Se aplican principios de arquitectura moderna como **serverless**, federación de identidad y segmentación por dominios funcionales, lo que permite escalar, auditar y evolucionar la solución sin comprometer la seguridad ni la experiencia del usuario.

16. Plan de Migración Gradual

La estrategia de migración propuesta para la entidad bancaria se basa en un enfoque incremental, gradual y controlado, que permite la coexistencia temporal entre el Core Bancario Tradicional y el sistema de banca por Internet, garantizando continuidad operativa y cumplimiento normativo durante todo el proceso.

Fase	Acciones Clave	Mecanismos de Mitigación de Riesgo	Resultado Esperado
1. Preparación	- Implementar ESB/API Gateway como capa de integración. - Segmentar por VPCs (Frontend, Backend, Seguridad, Datos, Core). - Configurar monitoreo y auditoría (CloudTrail, CloudWatch).	- Aislamiento de dominios. - Control de accesos con IAM granular. - Auditoría completa de eventos.	Entorno seguro y trazable para iniciar la migración.
2. Integración Progresiva	- Incorporar módulos del core digital vía APIs y eventos. - Redirigir gradualmente tráfico desde el core tradicional. - Validar seguridad y funcionalidad en cada dominio.	- Pruebas controladas por fases. - Rollback disponible en cada etapa. - Monitoreo de latencia y SLA.	Servicios digitales operando en paralelo sin afectar continuidad.
3. Coexistencia Operativa	- Ejecutar ambos cores bajo modelo multicore. - Sincronizar datos con colas/eventos (Kafka/SQS). - Ajustar rendimiento y escalabilidad.	- Alta disponibilidad (HA) con balanceadores. - Replicación en RDS Multi-AZ. - Auto-healing en instancias.	Operación estable con transición gradual hacia el core digital.
4. Retiro del Core Legacy	- Migrar funcionalidades críticas al core digital. - Activar mecanismos de rollback y recuperación (S3, RDS). - Validar cumplimiento normativo (LOPD, PCI DSS, ISO 27001).	- Backups y replicación. - Documentación técnica y legal. - Monitoreo continuo de seguridad.	Core digital como sistema principal, con riesgo operativo minimizado.

17.Conclusiones

Vamos a citar las siguientes conclusiones respecto a la arquitectura propuesta del sistema bancario por Internet :

1. **Arquitectura Modular y Escalable:** El diseño basado en microservicios y patrones como **CQRS**, **Event Sourcing** y **Saga** asegura que el sistema pueda crecer de manera controlada, manteniendo resiliencia y consistencia en operaciones críticas como pagos y transferencias.

- Justificación:
 - Escalabilidad horizontal con despliegue independiente de servicios.
 - Resiliencia ante fallos gracias a patrones distribuidos y recuperación de estado.

2. **Seguridad Multicapa y Autenticación Biométrica** : La incorporación de **API Gateway, OAuth2.0 + PKCE y biometría facial** fortalece la seguridad del ecosistema. El flujo de reconocimiento facial garantiza una experiencia de usuario personalizada y confiable, cumpliendo estándares internacionales de protección de datos.
 - Justificación:
 - Cumplimiento normativo (ISO/IEC 30107, FIDO2, LOPDP, GDPR).
 - Reducción de riesgos de acceso no autorizado mediante autenticación multifactor y biometría.
3. **Integración y Gobernanza de APIs**: El modelo de gobernanza de APIs y microservicios con **API Manager y Service Mesh** asegura orden, trazabilidad y evolución controlada. La estrategia de versionamiento y seguridad permite compatibilidad con sistemas existentes y protección contra amenazas.
 - Justificación:
 - Facilita integración con fintechs y reguladores bajo estándares de Open Finance.
 - Mejora la observabilidad y auditoría con trazabilidad completa de llamadas.
4. **Persistencia y Auditoría Confiables**: La aplicación de **CQRS + Event Sourcing** garantiza trazabilidad total y auditoría regulatoria. El registro de acciones y almacenamiento optimizado para clientes frecuentes mejora la experiencia de usuario y facilita el cumplimiento normativo.
 - Justificación:
 - Auditoría verificable para reguladores financieros (PCI DSS, ISO 27001).
 - Optimización de rendimiento con cache Redis para clientes frecuentes.
5. **Notificaciones Redundantes y Entrega Garantizada**: La integración con múltiples proveedores (**Twilio, Firebase, SES/SNS**) asegura redundancia y confiabilidad en la entrega de alertas críticas. El uso de **Outbox Pattern y DLQ** refuerza la garantía de entrega y la trazabilidad de eventos.
 - Justificación:
 - Redundancia multicanal para asegurar comunicación con clientes.
 - Cumplimiento de normativas de continuidad operativa (ISO 22301).
6. **Infraestructura en Nube Resiliente**: La evaluación entre **AWS y Azure** muestra que la nube es el pilar de la escalabilidad y continuidad operativa. La estrategia multi-cloud permite aprovechar lo mejor de cada proveedor, garantizando alta disponibilidad, tolerancia a fallos y recuperación ante desastres.
 - Justificación:

- Multi-región para resiliencia y baja latencia.
 - Optimización de costos con servicios serverless y contenedores.
7. **Experiencia de Usuario y Cumplimiento Normativo:** El sistema está diseñado para reducir fricción en el acceso, mejorar la experiencia del cliente y cumplir con normativas internacionales (**PCI DSS, ISO 27001, GDPR, LOPDP**). Esto asegura confianza tanto para usuarios como para reguladores.
- Justificación:
 - Flujo intuitivo y biometría para mejorar la experiencia del cliente.
 - Cumplimiento normativo que garantiza seguridad jurídica y reputacional.

18. Glosario de términos y siglas

- **BIAN (Banking Industry Architecture Network):** Estándar internacional que define dominios y servicios de referencia para arquitecturas bancarias modernas.
- **HA (High Availability):** Alta disponibilidad; capacidad de un sistema para estar operativo 24/7 sin interrupciones.
- **DR / DRP (Disaster Recovery / Disaster Recovery Plan):** Recuperación ante desastres; conjunto de estrategias y procedimientos para restaurar servicios tras fallos graves.
- **OAuth2.0:** Protocolo estándar de autorización que permite acceso seguro a recursos sin compartir credenciales directamente.
- **OpenID Connect:** Extensión de OAuth2.0 que añade autenticación de identidad.
- **KYC (Know Your Customer):** Normativa que obliga a verificar la identidad de los clientes para prevenir lavado de dinero.
- **AML (Anti-Money Laundering):** Conjunto de regulaciones contra el lavado de dinero.
- **GDPR (General Data Protection Regulation):** Reglamento europeo de protección de datos personales.
- **Habeas Data:** Derecho constitucional en varios países (incluido Ecuador) que protege la privacidad de datos personales.
- **ISO 27001:** Estándar internacional de gestión de seguridad de la información.
- **PCI-DSS (Payment Card Industry Data Security Standard):** Normativa de seguridad para el manejo de datos de tarjetas de pago.
- **ISO 20022:** Estándar internacional de mensajería financiera para pagos y transferencias.
- **SPA (Single Page Application):** Aplicación web que carga una sola página y actualiza dinámicamente su contenido.
- **API (Application Programming Interface):** Conjunto de reglas que permiten la comunicación entre sistemas.
- **API Gateway:** Punto central de entrada para gestionar, asegurar y monitorear APIs.
- **Apigee:** Plataforma de Google para gestión y gobierno de APIs.

- **MSK (Managed Streaming for Kafka):** Servicio administrado de AWS para ejecutar Apache Kafka.
- **Kafka:** Plataforma de mensajería distribuida orientada a eventos.
- **CQRS (Command Query Responsibility Segregation):** Patrón que separa operaciones de lectura y escritura en sistemas.
- **Event Sourcing:** Patrón que guarda eventos históricos en lugar de estados actuales, garantizando trazabilidad.
- **Saga Pattern:** Patrón para manejar transacciones distribuidas en microservicios con pasos compensatorios en caso de fallo.
- **Adapter Pattern:** Patrón de diseño que desacopla la lógica de negocio de servicios externos mediante adaptadores.
- **Factory Pattern:** Patrón de diseño que centraliza la creación de objetos complejos.
- **Service Mesh (ej. Istio):** Infraestructura que gestiona comunicación entre microservicios, añadiendo seguridad, balanceo y observabilidad.
- **EKS / AKS / OpenShift:** Plataformas de Kubernetes administradas por AWS, Azure y Red Hat respectivamente.
- **CI/CD (Continuous Integration / Continuous Deployment):** Prácticas de DevOps para integrar y desplegar software de forma continua y automatizada.
- **DDD (Domain-Driven Design):** Enfoque de diseño que organiza la arquitectura en torno a dominios del negocio.
- **CEP (Complex Event Processing):** Procesamiento de eventos complejos en tiempo real para detectar patrones.
- **SNS (Simple Notification Service):** Servicio de AWS para envío de notificaciones multicanal.
- **Pinpoint:** Servicio de AWS para mensajería dirigida y notificaciones móviles.
- **CloudWatch:** Servicio de AWS para monitoreo y trazabilidad.
- **GuardDuty / Security Hub:** Servicios de AWS para detección de amenazas y gestión de seguridad.
- **IAM (Identity and Access Management):** Servicio de AWS para gestión de identidades y permisos.
- **RDS (Relational Database Service):** Servicio de AWS para bases de datos relacionales administradas.
- **DynamoDB:** Base de datos NoSQL administrada por AWS.
- **Terraform / CloudFormation:** Herramientas de infraestructura como código para desplegar y gestionar recursos cloud.